

Recurrent Looped Transformer

Yifan Zhang Jichen Feng Shihan Qin

yifanzhangresearch@gmail.com

September 12, 2026*

Abstract

Recurrent Looped Transformer (RLT) passes a decoder’s final hidden state to the next token, together with that token’s causal encoder representation. The decoder also retains a sliding-window attention (SWA) cache at each layer. It updates both forms of state for every prompt and response token, so the recurrent computation continues across the prompt–response boundary. The path through successive states grows with sequence length, while the number of blocks evaluated per token stays fixed. We describe how to batch encoder work and independent decoder sequences, and how to replay a sampled history under current parameters for reinforcement learning (RL). Replay uses the same state transition as sampling and rebuilds both hidden states and caches after a parameter update. Preliminary eight-layer comparisons with a decoder-only Transformer show earlier parity learning for some RLT layouts and higher accuracy on flat modular arithmetic, with less separation on bracketed arithmetic and low accuracy on standard permutation tracking. Parity uses three initialization seeds; the other tasks use one.

Project Page: <https://github.com/yifanzhang-pro/recurrent-looped-tranformer>

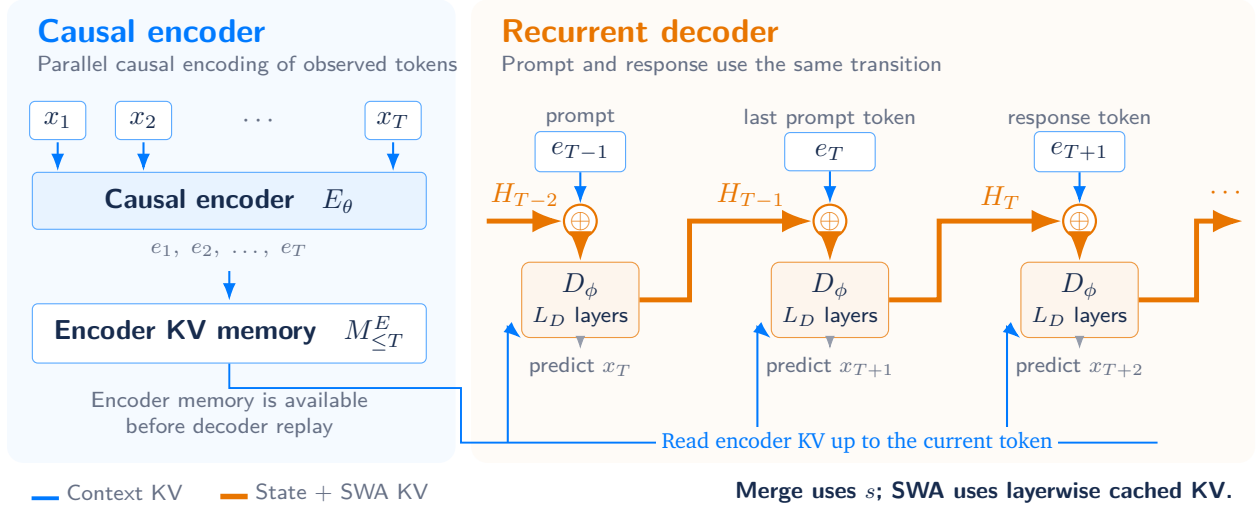


Figure 1 Recurrence across prompt and response. The encoder builds causal key–value (KV) memory; the decoder processes tokens in order. Orange arrows carry the decoder state $H_t = (s_t, C_t^D)$: the merge reads s_t , and each SWA layer reads its own cache. Blue arrows supply encoder memory up to the current position. The drawing shows the last two prompt updates and the first response update.

*Revised: September 15, 2026

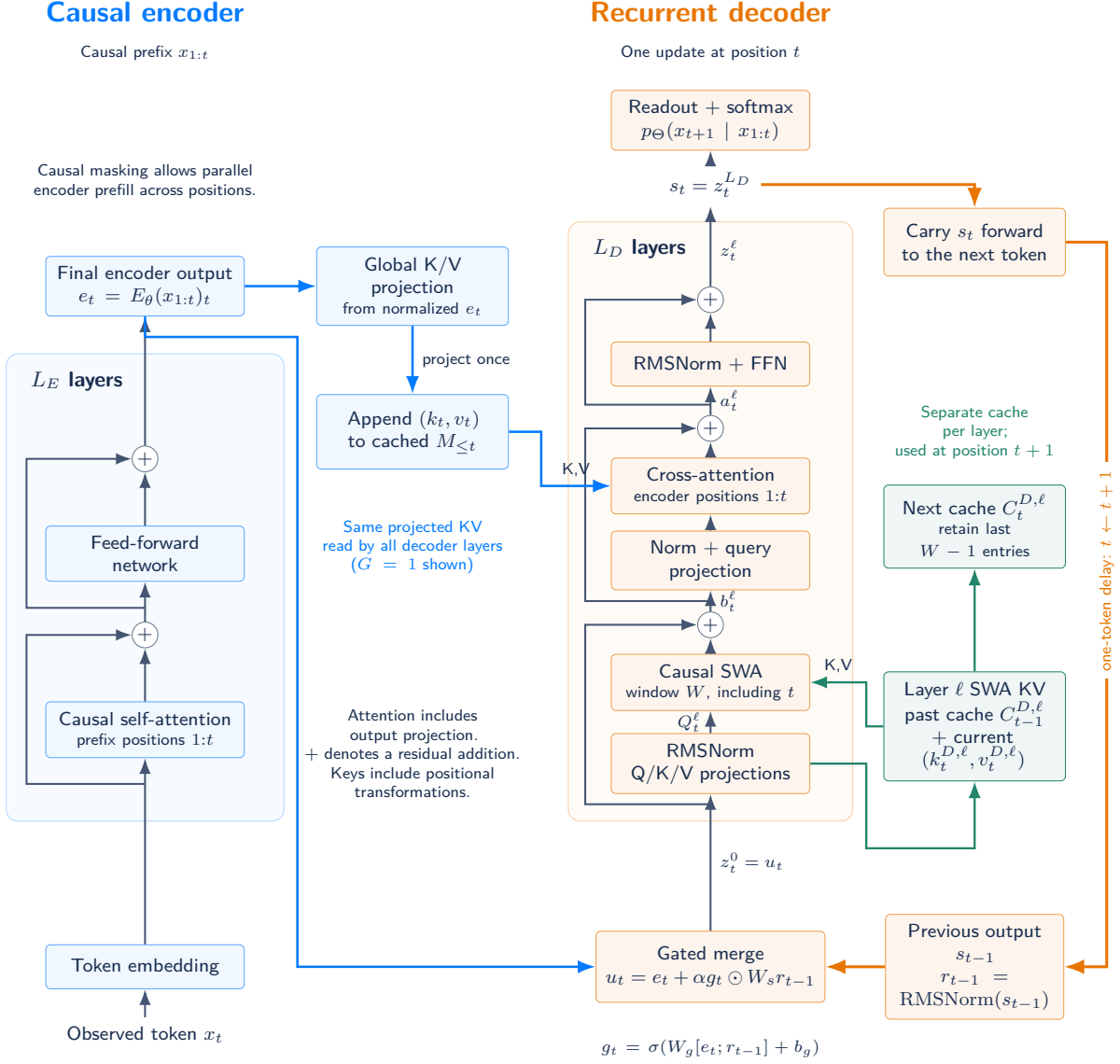


Figure 2 RLT architecture for one token update. Computation flows upward through the encoder and decoder stacks; each outlined stack repeats for its indicated depth. The encoder output supplies both the gated merge and the global KV projection. The drawing uses $G = 1$: all decoder layers read the same cached, projected global KV with their own queries. Each decoder layer constructs separate SWA KV from its own input, attends to its local window, then reads global memory and applies an FFN. Residual paths bypass each attention or FFN sublayer. The final decoder output s_t feeds the next token’s merge, while each layer retains its own SWA cache. Blue arrows carry encoder features and global memory, green arrows carry local KV, and orange arrows carry recurrent hidden states.

1 Introduction

Recurrent Looped Transformer (RLT) combines a causal encoder with a recurrent decoder made of Transformer blocks. The encoder builds memory from the token history. At each position, the decoder merges the current encoder representation with its own previous output, attends to encoder memory and recent decoder activations, and predicts the next token. The same update runs over the prompt and the response.

This design carries a continuous hidden state between tokens. After t tokens, the recurrent path passes through tL_D decoder blocks, where L_D is the decoder depth. Longer sequences therefore allow longer computation paths without adding learned layers. Compatible encoder and decoder weights can also be shared. For example, a tied 48-layer encoder and 48-layer decoder use two logical passes through shared attention and feed-forward weights, with 96 block evaluations per token.

Recurrence imposes a scheduling cost: decoder positions must run in order, including during prompt prefill. Encoder positions can run in parallel, and independent sequences can contribute decoder updates to the same batch. We describe how this separation allows weight and memory reuse, along with activation checkpointing for training.

Training and sampling must also agree on how the state is built. For RL, this means replaying the prompt and sampled response under the current parameters before evaluating action probabilities. The sampler’s recorded probabilities still describe the policy that generated those actions. We specify the forward replay and gradient paths needed to keep this distinction explicit after a parameter update.

RLT draws on encoder-memory reuse in YOCO (Sun et al., 2024) and temporal feedback in Feedback Transformer, Recurrent Transformer, Full-bandwidth Transformer, T²MLR, and Latent Recurrent Transformer (Fan et al., 2020; Oncescu et al., 2026; Wang et al., 2026; Cai et al., 2026; Huang et al., 2026). We focus on feedback through the full decoder, its execution schedule, and its use in policy replay. The experiments compare eight-layer layouts on addition, parity, modular arithmetic, and permutation state tracking.

2 Method

2.1 Sequence and state

Let $x_{1:S}$ be an independent sequence beginning with $x_1 = \text{BOS}$. At inference, $x_{1:T}$ is the prompt and later tokens form the response. The model uses the same conditional distribution on either side of T . Let L_E and L_D be the encoder and decoder depths, and let d be the residual width. We write the encoder representation $e_t \in \mathbb{R}^d$ and recurrent output $s_t \in \mathbb{R}^d$ as column vectors.

The complete decoder state is $H_t = (s_t, C_t^D)$. The cache C_t^D holds the key/value projections retained by each decoder SWA layer. The window size $W \geq 1$ includes the current token, so each layer retains at most $W - 1$ past positions after an update. We collect all parameters, including the learned initial state s_* , in Θ . The encoder and decoder are denoted by E_θ and D_ϕ .

2.2 Causal encoder and memory

For an observed prefix, compute

$$e_{1:T} = E_\theta(x_{1:T}). \quad (2.1)$$

Positions within each encoder layer can be processed together using a causal mask. Encoder layers remain sequential. For decoder memory group $g \in \{1, \dots, G\}$, construct

$$k_t^g = \mathcal{P}_K^g(e_t, t), \quad v_t^g = W_V^g \text{RMSNorm}_E(e_t), \quad M_{\leq t}^g = \{(k_j^g, v_j^g)\}_{j=1}^t. \quad (2.2)$$

The key map includes normalization, projection, and any positional transformation. Decoder layer ℓ reads group $g(\ell)$: $G = 1$ shares memory across layers, while $G = L_D$ allows separate projections for each layer. The decoder reads two stores through separate attention sublayers. Cross-attention reads encoder memory $M_{\leq t}$ over the full prefix; causal SWA reads decoder KV within a bounded window.

2.3 One transition for every token

Initialize once, before BOS:

$$H_0 = (s_\star, \emptyset). \quad (2.3)$$

For every observed or sampled token, apply

$$u_t = \text{Merge}(e_t, s_{t-1}), \quad (2.4)$$

$$H_t = (s_t, C_t^D) = D_\phi(u_t; M_{\leq t}, C_{t-1}^D, t), \quad t \geq 1, \quad (2.5)$$

$$p_\Theta(x_{t+1} \mid x_{1:t}) = \text{softmax}(W_o \text{RMSNorm}_o(s_t))_{x_{t+1}}. \quad (2.6)$$

Define $F_t(H) = D_\phi(\text{Merge}(e_t, s); M_{\leq t}, C^D, t)$ for $H = (s, C^D)$. The encoder features can be computed before the decoder runs. To predict the first response token, the decoder must then process every prompt position:

$$H_T = F_T \circ F_{T-1} \circ \dots \circ F_1(H_0). \quad (2.7)$$

Generation continues from H_T . After sampling x_{T+1} from s_T , encode it incrementally, append its encoder-derived KV, and compute $H_{T+1} = F_{T+1}(H_T)$, including every decoder SWA cache update. Neither component of H_T is reset at the serving boundary.

2.4 Prompt prefill and incremental decoding

Prefill first computes the causal encoder representations and memory of the prompt, then evaluates $H_1, \dots, H_T$ in order. At decoder position t , attention is restricted to $M_{\leq t}$ even though the entire prompt memory is available. Generation uses

$$(e_t, C_t^E) = E_\theta^{\text{step}}(x_t, C_{t-1}^E) \quad (2.8)$$

with encoder cache C^E , followed by a memory append and decoder update. Observed tokens may be encoded in chunks if the encoder cache is preserved. The decoder still processes every token in order: each update produces the hidden state and SWA KV needed by later positions.

2.5 Merge and decoder blocks

We use the following gated merge:

$$r_{t-1} = \text{RMSNorm}_s(s_{t-1}), \quad (2.9)$$

$$g_t = \sigma(W_g[e_t; r_{t-1}] + b_g), \quad (2.10)$$

$$u_t = e_t + \alpha g_t \odot W_s r_{t-1}. \quad (2.11)$$

Here $W_g \in \mathbb{R}^{d \times 2d}$, $W_s \in \mathbb{R}^{d \times d}$, and α controls the feedback scale. The experiments use $\alpha = 0.1$. A scalar gate or low-rank W_s would reduce the merge cost.

Starting from $z_t^0 = u_t$, each decoder block applies causal SWA, encoder-memory cross-attention, and a feed-forward network (FFN):

$$q_t^{D,\ell} = \mathcal{P}_Q^{D,\ell}(z_t^{\ell-1}, t), \quad (2.12)$$

$$k_t^{D,\ell} = \mathcal{P}_K^{D,\ell}(z_t^{\ell-1}, t), \quad v_t^{D,\ell} = W_V^{D,\ell} \text{RMSNorm}_{S,\ell}(z_t^{\ell-1}), \quad (2.13)$$

$$b_t^\ell = z_t^{\ell-1} + \text{Attn}_\ell^D \left(q_t^{D,\ell}, \{(k_j^{D,\ell}, v_j^{D,\ell})\}_{j=\max(1,t-W+1)}^t \right), \quad (2.14)$$

$$a_t^\ell = b_t^\ell + \text{Attn}_\ell^M \left(\mathcal{P}_Q^{M,\ell}(b_t^\ell, t), M_{\leq t}^{g(\ell)} \right), \quad (2.15)$$

$$z_t^\ell = a_t^\ell + \text{FFN}_\ell(\text{RMSNorm}_{D,\ell}(a_t^\ell)), \quad s_t = z_t^{L_D}. \quad (2.16)$$

The attention operators include their output projections; query/key maps include their respective normalizations and positional transformations. At each layer, current KV is formed before SWA, using the layer input, so current-position attention introduces no circular dependency. Historical decoder KV comes from C_{t-1}^D . After the update, retain positions $\max(1, t - W + 2), \dots, t$ in C_t^D ; this set is empty for $W = 1$. Position metadata follows the same convention during prefill, sampling, and replay. Alternative sublayer orders define different variants and must be used consistently in all execution modes.

Figure 2 expands the encoder and decoder stacks for one token update.

2.6 Sharing encoder and decoder weights

The reference tied RLT sets $L_E = L_D = L$. Encoder self-attention at layer ℓ and decoder SWA at layer ℓ share compatible query, key, value, and output projection matrices; their FFNs are also shared. The encoder uses its causal context, whereas decoder SWA uses decoder activations within its window. Decoder cross-attention has separate query/output projections and the memory projections of Equation (2.2). The encoder and decoder retain separate normalizations; the merge and readout are also separate modules. Cross-attention adds computation to the shared attention and FFN.

The two passes share weights but compute separate activations. An untied E_θ, D_ϕ uses separate weights and keeps the same recurrence. Memory groups can be shared in either version; each decoder layer still maintains its own SWA cache.

3 Computational Properties

3.1 Prompt–response consistency

Proposition 3.1 (Invariance to the serving split). Fix the parameters, token sequence, position convention, and initial state for an independent sequence. Assume mathematically equivalent causal encoder execution, identical SWA windows and cache updates, and deterministic decoder operations. Processing a prefix with batched encoder prefill followed by recurrent decoder updates gives the same states and next-token distributions as processing it incrementally. The conditional distribution for a fixed token history is therefore unchanged when the prompt–response split moves.

Proof. Causal encoder equivalence gives the same e_t and $M_{\leq t}$ in both schedules. Both start from $H_0 = (s_*, \emptyset)$. If their states agree at $t - 1$, Equation (2.5) applies the same operations to the same inputs at t , so their states agree at t . The result follows by induction, since the transition does not depend on the serving split. $\square$

3.2 Prefill work and sequential depth

Let $C_E^{\text{pf}}(T)$ be encoder prefill work, $C_M(T)$ the memory projection work, and $C_D^{\text{step}}(t)$ a decoder evaluation over t encoder-memory entries and at most W decoder positions per layer, including merge overhead. Then

$$C_{\text{prefill}}(T) = C_E^{\text{pf}}(T) + C_M(T) + \sum_{t=1}^T C_D^{\text{step}}(t). \quad (3.1)$$

For dense attention and width-proportional KV, a coarse arithmetic estimate is

$$O((L_E + L_D)(Td^2 + T^2d) + GTd^2 + L_DT \min(W, T)d). \quad (3.2)$$

Prefill requires T sequential decoder transitions, each containing L_D blocks. This dependency can reduce hardware utilization even when the arithmetic complexity has the same order as a dense Transformer.

Each new token evaluates $L_E + L_D$ blocks, plus merge and memory projection. Attention work still grows with context length. With effective KV width d_{KV} , inference cache storage is approximately

$$O((L_E + G)t d_{\text{KV}} + L_D \min(t, W - 1)d_{\text{KV}}^D + d). \quad (3.3)$$

Here d_{KV}^D is the effective decoder SWA KV width, and the $O(d)$ term stores the current recurrent output. The SWA term counts retained history; current-position KV and training activations require additional storage. Weight sharing reduces parameter storage, while both passes and their caches remain necessary.

3.3 Depth of the recurrent path

After a prompt of length T and n processed response tokens, the recurrent path has passed through $(T + n)L_D$ decoder blocks. Figure 3 compares this growing path with the fixed block count per token. The influence of earlier states depends on the learned gates, projections, and products of transition Jacobians.

4 Hardware Execution

4.1 Batching independent sequences

For a known training sequence or prompt, the encoder computes features and memory projections across positions in parallel. The decoder then processes positions in order. As Figure 4 shows, independent sequences can contribute their next decoder update to one batch. Each sequence keeps its own encoder prefix, SWA cache, recurrent output, and position.

At inference, each step consists of an encoder update, memory append, merge, decoder pass, and readout. Batching requests increases matrix-operation sizes and allows weight reuse. Small batches and uneven sequence lengths may limit utilization.

Block count per token and along the recurrent path

Illustrative 48-layer encoder + 48-layer decoder; counts exclude merge and readout.

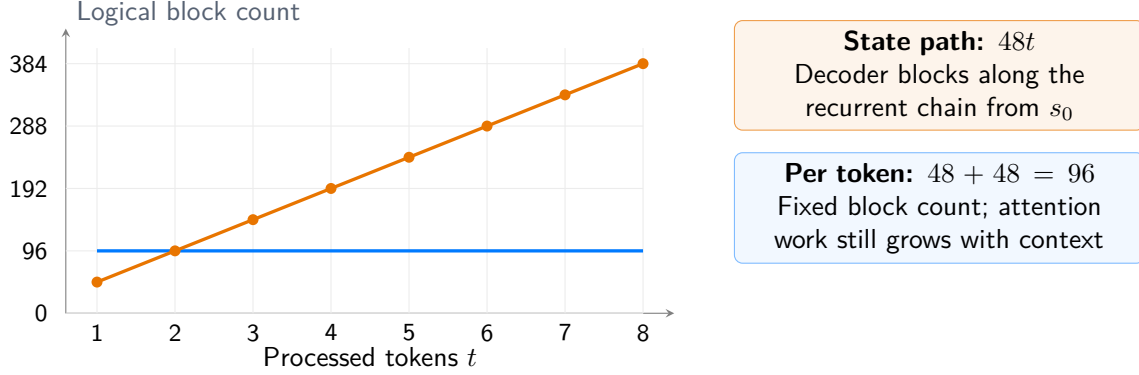


Figure 3 Recurrent path length and block count per token. For $L_E = L_D = 48$, the path traverses $48t$ decoder blocks after t tokens. Each token evaluates 96 encoder and decoder blocks. Orange counts decoder blocks along the recurrent path; blue counts blocks in both stacks per token.

Batching decoder updates from independent sequences

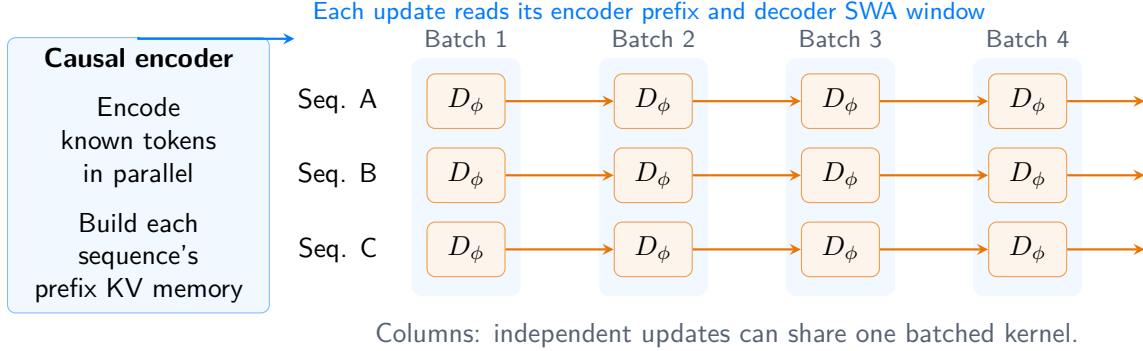


Figure 4 Batching decoder updates across sequences. Columns group independent updates into a batch; rows follow each sequence in token order. Each update reads its own encoder prefix and decoder SWA window. Known tokens can be encoded in parallel before replay; generated tokens are encoded as they arrive.

4.2 Memory traffic and parameter reuse

Encoder KV stays fixed while the prefix and parameters are unchanged. Decoder layers reuse that memory and append their own KV to separate SWA caches. Fewer memory groups reduce KV storage and projection work; more groups allow different transformations for each layer. Sharing encoder and decoder weights may also help keep weights resident on the accelerator.

One proposed optimization is to fuse normalization, gating, state projection, and residual addition while preserving their dependencies and numerical behavior. Attention kernels must preserve the encoder-prefix mask and local SWA window.

4.3 Training memory and numerical agreement

Activation checkpointing reduces stored activations by recomputing them during backpropagation through time (BPTT). Batch size and checkpoint placement depend on sequence length and available accelerator memory. Full BPTT retains gradients through the entire recurrent history.

Zhang et al. (2026) describe how the executed policy depends on precision, cache construction, reductions, and sampling transforms as well as weights. Sampler and trainer implementations therefore need consistent positional conventions, stochastic behavior, and cache contents, with numerical checks across execution modes.

5 Training Objectives

5.1 Autoregressive pretraining

Full-sequence next-token prediction is the base objective:

$$\mathcal{L}_{\text{PT}}(\Theta) = -\mathbb{E}_{x_{1:S}} \left[\frac{1}{S-1} \sum_{t=1}^{S-1} \log p_{\Theta}(x_{t+1} \mid x_{1:t}) \right]. \quad (5.1)$$

Initialize $H_0 = (s_*, \emptyset)$, compute causal encoder features, and update the decoder over positions $1, \dots, S-1$. Every non-BOS target contributes to the loss; the encoder, memory projections, merge, and decoder train jointly. At each independent document, reset both decoder and encoder state, reset positions, and prevent attention across document boundaries. When training on segments, specify the initial context and state: dropping an earlier state changes the history on which the likelihood is conditioned.

5.2 Supervised fine-tuning

Let $m_{t+1} = 1$ for assistant targets and 0 for user, system, tool, or padding targets. For examples with at least one selected target, optimize

$$\mathcal{L}_{\text{SFT}}(\Theta) = -\mathbb{E}_x \left[\frac{1}{\sum_{t=1}^{S-1} m_{t+1}} \sum_{\substack{1 \leq t < S \\ m_{t+1}=1}} \log p_{\Theta}(x_{t+1} \mid x_{1:t}) \right]. \quad (5.2)$$

The mask selects loss terms only. User, system, and tool tokens still update the decoder state, and gradients from assistant losses flow through those updates, including encoder memory and decoder KV. The model continues from the existing state when an assistant turn begins.

5.3 Policy gradients and replay

Let $c = x_{1:T}$ be a prompt and $y = (y_1, \dots, y_N)$ a sampled response, with $y_i = x_{T+i}$. The policy is

$$\pi_{\Theta}(y \mid c) = \prod_{i=1}^N p_{\Theta}(y_i \mid c, y_{<i}). \quad (5.3)$$

For a sequence reward $R(c, y)$ independent of Θ , define $J(\Theta) = \mathbb{E}_{c, y \sim \pi_{\Theta}}[R(c, y)]$. Its on-policy score-function gradient is

$$\nabla_{\Theta} J = \mathbb{E}_{c, y \sim \pi_{\Theta}} \left[(R(c, y) - b(c)) \sum_{i=1}^N \nabla_{\Theta} \log p_{\Theta}(y_i \mid c, y_{<i}) \right], \quad (5.4)$$

where $b(c)$ is a response-independent baseline, held constant when taking the policy gradient. During replay, the sampled tokens are fixed and gradients pass through the recurrent computation used to evaluate their log-probabilities.

Figure 5 shows the replay procedure. The sampler records each action’s log-probability under the distribution μ that sampled it. The trainer rebuilds encoder features, recurrent outputs, and decoder SWA caches under the current parameters Θ , starting from the initial state and processing the full prompt and sampled response prefix. The resulting action probabilities give the ratios

$$r_i(\Theta) = \exp(\log p_{\Theta}(y_i \mid c, y_{<i}) - \log \mu(y_i \mid c, y_{<i})). \quad (5.5)$$

Here the target is the raw model policy p_{Θ} . The behavior probabilities must include any temperature scaling, truncation, and renormalization used by the sampler. Exact importance sampling requires $p_{\Theta}(\cdot \mid h) \ll \mu(\cdot \mid h)$: at each relevant history, every action with positive target probability must also have positive behavior probability. Top- k or top- p sampling generally excludes actions with positive probability under an untruncated softmax target. If the target policy itself uses a sampling transform, apply it consistently in the objective and ratios.

The trainer must preserve both the forward probabilities and their parameter derivatives (Zhang et al., 2026). It therefore differentiates the recurrent computation or an equivalent implementation.

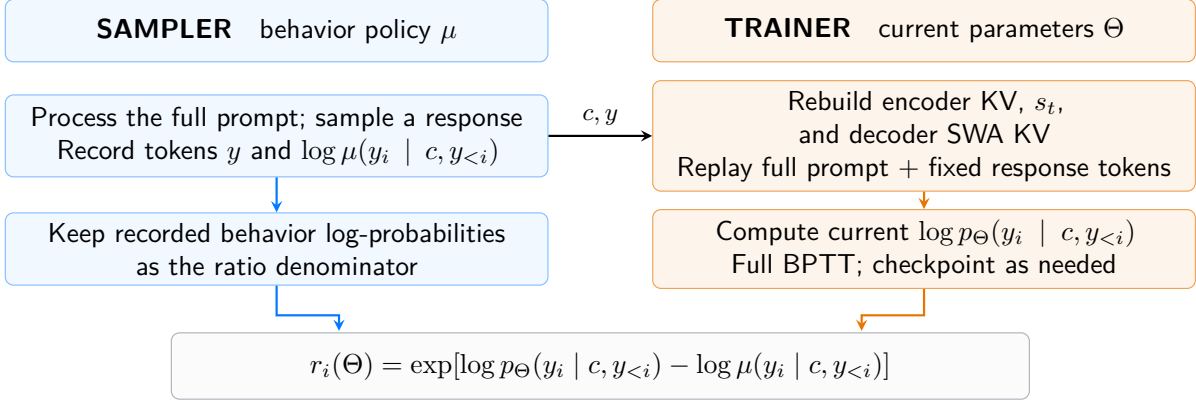
For fixed prompts and sequence rewards, exact trajectory importance sampling uses $\prod_i r_i$, subject to support and integrability conditions. Tokenwise clipping and other surrogate losses may introduce bias even with correct replay probabilities.

5.4 Gradients and cache reuse during training

Full BPTT follows gradients through encoder memory, recurrent outputs, and decoder SWA KV over the entire prompt and response. This remains true when only response tokens contribute to the loss. Recomputing the prompt under current parameters with `no_grad` gives the same forward values but drops the prompt-state derivatives.

The reference computation uses full BPTT, with activation checkpointing as needed. Checkpointing recomputes activations during backward using the same parameters and stochastic state. Truncated BPTT (TBPTT) instead detaches selected tensors at a boundary, keeping their values while cutting their gradient paths. Detaching only s_t leaves paths through decoder KV and encoder memory, so a truncation scheme must state which tensors it detaches.

Replaying a sampled history under current parameters



After every parameter update: rebuild current-policy KV and states; retain original behavior log-probabilities.

Figure 5 Replaying a sampled history under current parameters. The sampler records behavior probabilities. The trainer rebuilds the full prefix, including recurrent outputs and decoder SWA caches, to evaluate the current policy. The ratio compares the current action probability with the recorded behavior probability.

Keep parameters fixed throughout each replay and backward pass. After an optimizer update, rebuild encoder KV, recurrent outputs, and decoder SWA KV before the next exact replay. Recomputation may start from the sequence beginning or from an exact prefix checkpoint made with the current parameters and execution settings. The recorded behavior log-probabilities remain unchanged, since they describe the policy that sampled the actions.

6 Multi-turn Inference and Cache Reuse

A conversation forms one token history beginning at BOS. User messages, tool results, role delimiters, and assistant tokens all receive encoder and decoder updates. For example, when a tool returns, encode its output using the cached prefix, then process those tokens through the decoder in order before resuming generation. Reset state only for a new independent sequence or an explicitly defined context reset.

To resume from a saved prefix, retain the encoder cache C_t^E , cross-attention memory $M_{\leq t}$, decoder state $H_t = (s_t, C_t^D)$, tokens and positions, SWA window convention, and model version. These values can be reused at fixed weights regardless of where the prompt ends. Reproducing the same sampled outputs also requires the sampler’s random state. If only text is saved, replay it to rebuild the state.

Editing or truncating an earlier token invalidates the later cached state. Recompute from a valid checkpoint before the edit; this also applies to chat templates that rewrite earlier tokens. In multi-turn RL, user and tool tokens update the state and carry gradients, but receive no action importance-ratio factors because the policy did not sample them.

7 Preliminary Algorithmic Experiments

We compare encoder–decoder depth splits on addition, parity, modular arithmetic, and permutation state tracking. The results are a snapshot taken on September 15, 2026, from 16:11:47 to 16:12:10 UTC, while training was still running. At that point, 21 of 48 runs had reached the planned 2,000 optimizer steps. All scores use held-out validation examples at training lengths.

7.1 Models, tasks, and evaluation protocol

All models have eight logical layers, width 512, FFN width 1,365, and four attention heads. We compare untied RLT splits $4 + 4$, $5 + 3$, $6 + 2$, $7 + 1$, and $8 + 0$ with an eight-layer decoder-only Transformer. RLT uses an SWA window of eight, one shared encoder-memory group, and feedback scale $\alpha = 0.1$. The $8 + 0$ variant has no decoder blocks but still applies the gated recurrent merge. RLT has 26.10–28.73M parameters, compared with 25.31M for the Transformer (Table 3).

A $4 + 4$ RLT *without hidden-state feedback* control (RLT w/o feedback) was implemented after this snapshot for future ablations. It keeps the encoder, decoder SWA, and prefix-restricted memory attention while removing hidden-state feedback. It can process known tokens in parallel within each decoder layer; generation remains autoregressive. Figure 6 shows the architecture and its parallel execution over known tokens. Appendix D gives its configuration and implementation details.

All runs use global batch 512, microbatch 32, AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay 0.1, and gradient clipping at norm 1. The base learning rate rises to 10^{-4} over 200 steps, then follows cosine decay to 5×10^{-6} at step 2,000. Both architectures use the same width- μ P recipe and run in CPU FP32 with four threads. RLT uses a TBPTT interval of 128 tokens, which covers every training sequence in these tasks and therefore cuts no gradients here. Validation runs every 100 steps. The full budget is 1,024,000 training examples per run.

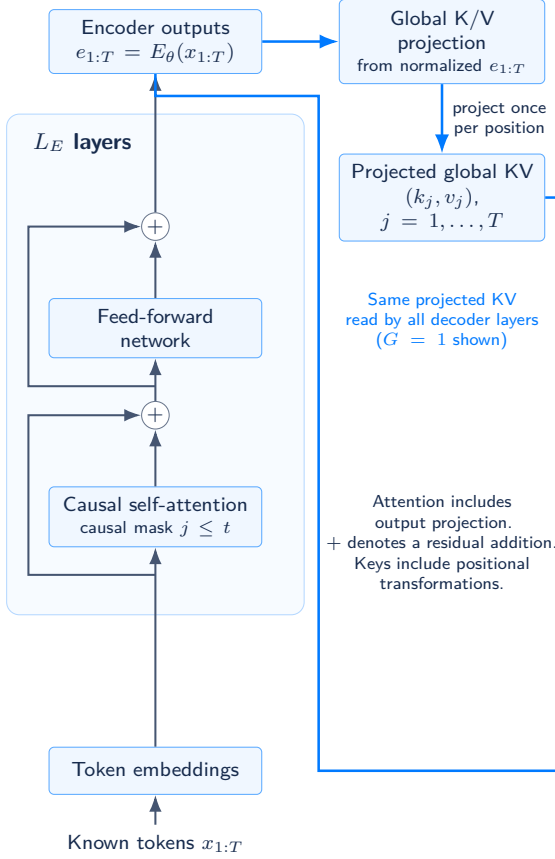
- **Addition:** randomly sample 1–8-digit operands and serialize their digits in reverse order. The 256 validation examples use the same range. An answer is correct only if greedy generation produces the entire answer and EOS correctly.
- **Parity:** train on binary strings of lengths 3–40 and supervise the final parity bit. Validation pools 256 examples at each of lengths 32, 36, and 40.
- **Modular arithmetic:** evaluate expressions modulo five using $+$, $-$, and $\times$. The flat variant respects multiplication precedence and trains on odd expression lengths 3–39, with validation lengths 33, 35, and 39. The bracketed variant uses expression trees, trains on lengths 3–40, and validates at 32, 36, and 40. Each validation length has 256 examples; only the final value is supervised.
- **S_5 state tracking:** compose 32 permutations and supervise the running product after each input, following [Grazzi et al. \(2024\)](#). Standard inputs range over all 120 permutations; the swaps variant uses the identity and the ten single transpositions. Each variant has 256 validation sequences of length 32. We report final-state accuracy as the primary metric and prefix accuracy in the appendix.

Parity uses initialization seeds 42, 43, and 44; the other tasks use seed 42. Data-generation seeds, training examples, and validation sets are fixed across models and parity runs. For parity, we pool validation examples within each run and report the mean and sample standard deviation

Causal encoder

Known sequence $x_{1:T}$

Causal masking allows parallel encoder prefill across positions.



Decoder w/o feedback

Parallel positions within each layer

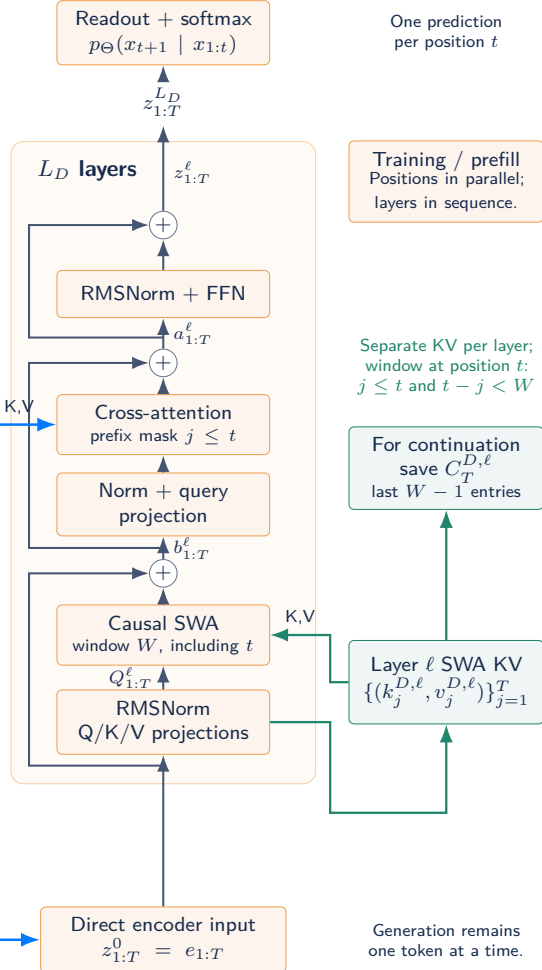


Figure 6 RLT w/o feedback during training or prompt prefill. The decoder receives the encoder outputs directly, $z_{1:T}^0 = e_{1:T}$, with the gated merge and final-hidden-state feedback path removed. Each outlined stack repeats for its indicated depth. For $G = 1$, all decoder layers read the same projected global KV; query position t can read encoder positions $j \leq t$. Each decoder layer constructs its own SWA KV and applies a causal window of size W , including the current position. Known positions run together within each layer, while layers follow depth order. Incremental generation retains the encoder cache, global memory, and each layer's last $W - 1$ SWA entries and proceeds one token at a time.

Table 1 Validation accuracy (%) at the latest step shared by all runs within each task. All six models in a row have consumed the same number of examples. Parity is mean $\pm$ sample SD over three initialization seeds; other entries use one seed.

Task	Step	RLT 4+4	RLT 5+3	RLT 6+2	RLT 7+1	RLT 8+0	GPT 8
Addition	1000	100.00	98.44	100.00	100.00	100.00	100.00
Parity	500	83.29 $\pm$ 28.94	83.51 $\pm$ 28.00	99.44 $\pm$ 0.98	82.77 $\pm$ 29.84	48.70 $\pm$ 2.60	48.48 $\pm$ 0.53
Mod 5, no brackets	800	22.40	21.74	20.83	22.53	21.74	20.70
Mod 5, brackets	800	37.50	36.72	36.72	40.10	37.63	35.55
S_5 , swaps	1000	100.00	99.61	95.70	89.84	86.33	89.84
S_5 , standard	1000	1.17	3.52	1.95	1.56	1.17	1.17

Table 2 Completed mod-5 subset at 2,000 steps, seed 42. Every entry is validation accuracy (%) after 1,024,000 training examples. These three models had completed both variants at the snapshot; deeper-decoder allocations were still running.

Task	RLT 7+1	RLT 8+0	GPT 8
No brackets	95.44	90.89	20.57
With brackets	78.26	75.52	79.56

(SD) across the three initializations. The SD measures variation due to initialization on a fixed data stream.

Figure 7 plots the available validation checkpoints. Each parity mean uses all three seeds at the same step; the curve ends when any seed lacks a checkpoint. Table 1 compares all six models at the latest step available for every required run within each task. Table 2 separately compares the completed mod-5 runs at 2,000 steps.

7.2 Parity: learning speed and initialization variability

At 500 steps, RLT 6 + 2 reaches $99.44 \pm 0.98\%$ validation accuracy, compared with $48.48 \pm 0.53\%$ for the Transformer. RLT 4 + 4, 5 + 3, and 7 + 1 average about 83%, with SDs of 28–30 percentage points; 8 + 0 remains near chance. Figure 8 shows the individual seeds behind these averages. Because the validation set is fixed, their spread reflects different training outcomes across initializations.

All three RLT 6 + 2 seeds reach 100% by step 600. The Transformer learns more slowly in these runs, reaching $94.84 \pm 3.43\%$ at 2,000 steps.

7.3 Modular arithmetic with and without brackets

At step 800, all six models are near the 20% uniform reference on flat modular arithmetic. At 2,000 steps with seed 42, RLT 7 + 1 and 8 + 0 reach 95.44% and 90.89%, while the Transformer remains at 20.57% (Table 2).

On bracketed expressions, the Transformer reaches 79.56%, compared with 78.26% and 75.52% for those two RLT layouts. The task generators differ in operator structure and label distribution. Parentheses also consume positions, so equal token lengths can contain different numbers of arithmetic operations.

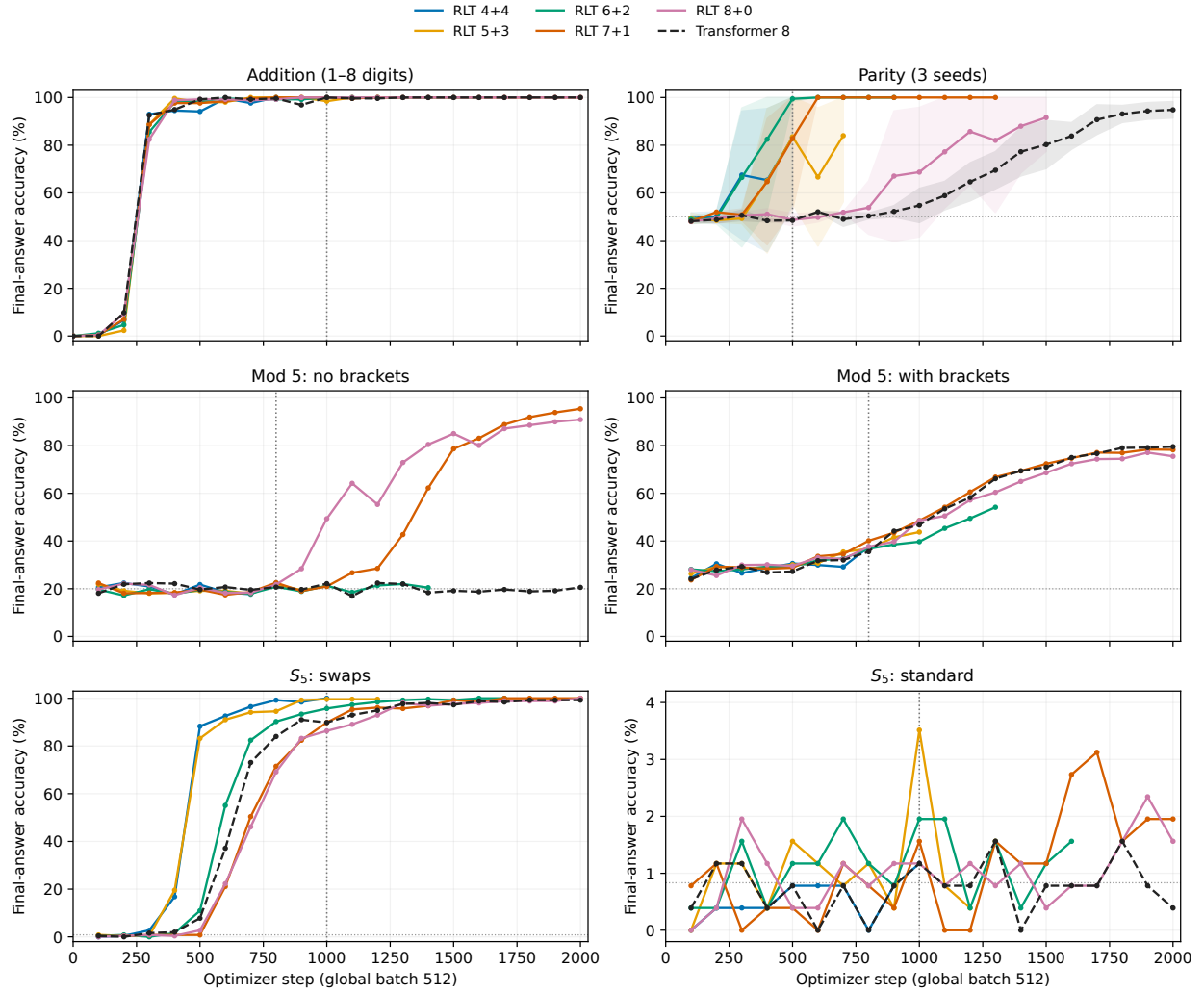


Figure 7 Validation accuracy during training. Addition requires a correct greedy answer and EOS; the other tasks measure final-answer or final-state accuracy. Parity shows the mean $\pm$ sample SD across three initialization seeds at every plotted step ($n = 3$), with bands clipped to the accuracy range. Other panels show one seed. Curves are unsmoothed and stop at the available checkpoints. Vertical dotted lines mark the steps used in Table 1; horizontal dotted lines show uniform-prediction accuracy. Standard S_5 uses a narrower vertical scale because its accuracies are low.

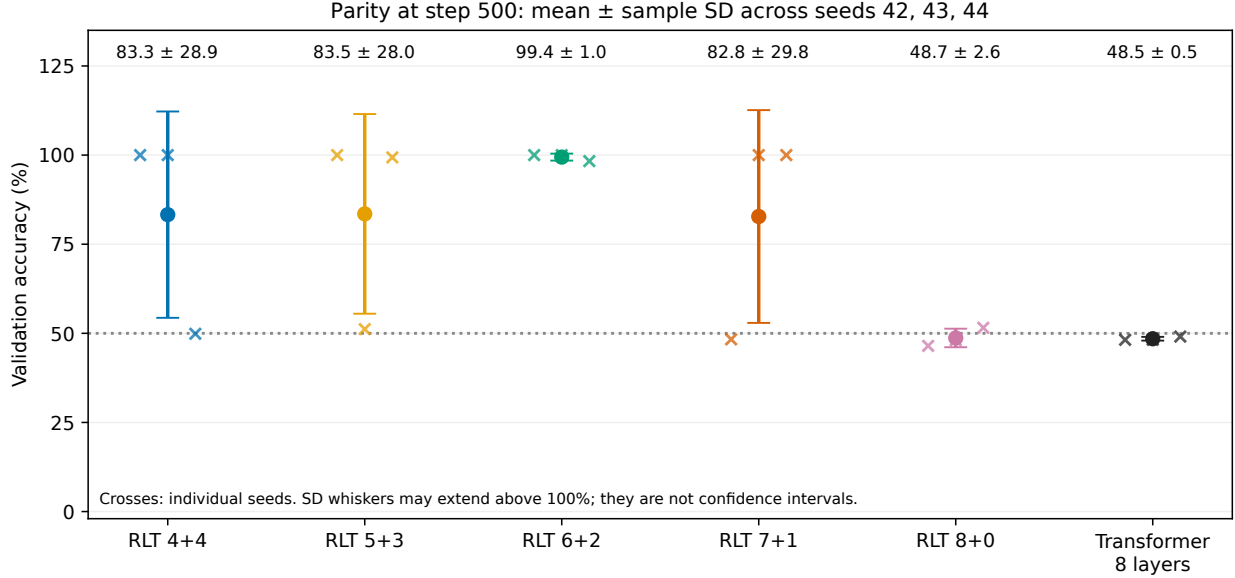


Figure 8 Parity at 500 training steps. Circles and whiskers show the mean and sample SD across seeds 42, 43, and 44; crosses show individual seeds with a small horizontal offset. SD whiskers describe the spread across seeds and may extend beyond 100%. Every model uses the same 768 validation examples and has seen 256,000 training examples per seed.

7.4 Addition and permutation state tracking

At 1,000 steps, every model reaches 100% addition accuracy except RLT 5 + 3, at 98.44%. The task therefore gives little separation at this checkpoint within the 1–8-digit training range.

On swaps- S_5 at the same step, RLT 4 + 4 and 5 + 3 reach 100% and 99.61% final-state accuracy, compared with 89.84% for the Transformer. The Transformer’s prefix-token accuracy is already 98.75%, so averaging over prefixes would obscure some errors at the end of the sequence. By 2,000 steps, the completed 7 + 1, 8 + 0, and Transformer runs reach 100%, 100%, and 99.22% final-state accuracy.

Standard S_5 remains difficult for all six models: scores at 1,000 steps range from 1.17–3.52%, against a uniform 120-class reference of 0.83%.

7.5 Training stability and comparison scope

Parity RLT 5 + 3 with seed 42 drops from 100% at step 500 to 48.05% at step 600, then returns to 100% at step 700. Figure 9 shows the training losses, and Appendix D gives the individual parity seed curves. Such regressions call for learning-rate and feedback ablations across more seeds; a shared optimizer recipe may suit the architectures differently.

The preferred encoder–decoder split varies by task. These comparisons change feedback, attention structure, parameter count, and compute together, so matched RLT w/o feedback runs are needed to isolate hidden-state feedback. The evaluation covers learning at training lengths, with three initializations for parity and one for each other task. Hardware throughput and RL performance remain to be measured.

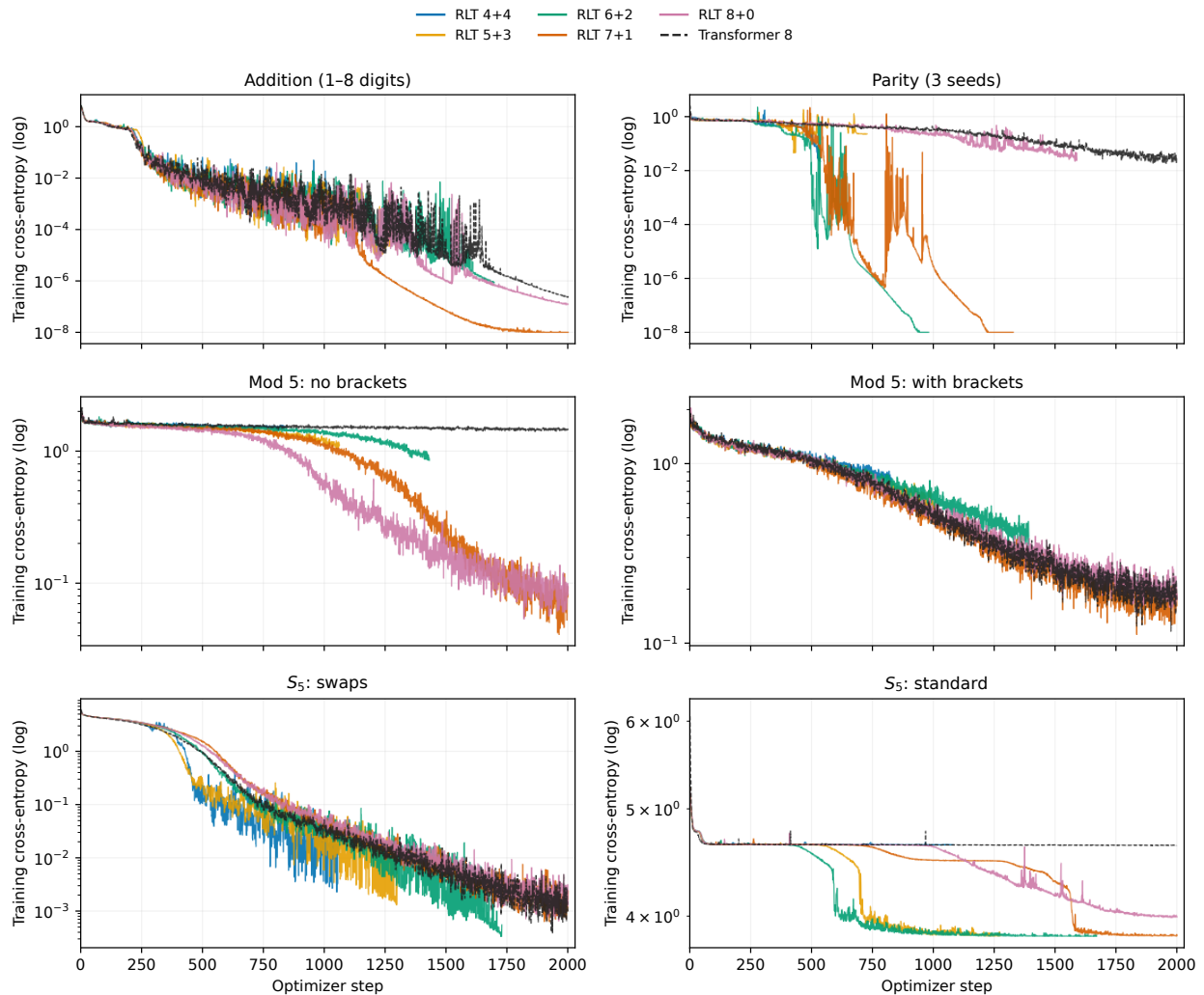


Figure 9 Unsmoothed training cross-entropy at every recorded step. Parity averages all three seeds at each available common step; other tasks show seed 42. Values below 10^{-8} are displayed at 10^{-8} on the logarithmic axis. Loss is measured over the global batch before the optimizer update; logged gradient norms are measured before clipping. Compare losses within each task, since supervision differs across tasks.

8 Related Work

Hybrid Transformer–RNN models. Chen et al. (2018) combine a Transformer encoder with an LSTM-based RNMT+ decoder for machine translation. RLT uses the same broad split between parallel encoding and recurrent decoding. Its decoder consists of Transformer blocks with encoder-memory cross-attention and SWA, and it processes both prompt and response tokens in a causal language model.

Encoder-derived memory. YOCO builds reusable KV memory for an upper cross-decoder and allows early exit during prefill (Sun et al., 2024). DeepSeek-V4.1-Flash projects global decoder KV from final encoder states and maintains separate decoder SWA (DeepSeek-AI, 2026). RLT also builds global memory from encoder states, but its recurrent decoder must process every prompt token.

Temporal feedback. Feedback Transformer makes past representations available to later computation (Fan et al., 2020). Recurrent Transformer constructs each layer’s persistent KV from that layer’s output and provides an exact tiling schedule to improve memory movement (Oncescu et al., 2026). RLT feeds the previous final decoder output into the next decoder input. Its global memory comes from the encoder and its local SWA KV from the decoder. Its recurrent dependency spans the full decoder, whereas Recurrent Transformer’s feedback is layerwise.

Cross-token latent feedback. Full-bandwidth Transformer combines the previous top-layer hidden state with the next token embedding through a gated linear unit, retaining the Transformer stack and KV cache (Wang et al., 2026). Its multi-pass training shifts hidden states between passes to allow token-parallel teacher forcing, with a prefix mixin to address differences between prompts and generation. RLT places this kind of output-to-input feedback at the encoder–decoder interface and trains by replaying the decoder sequentially over the full history.

T²MLR feeds a cached middle-layer representation from the previous token into an earlier layer at the current position (Cai et al., 2026). Its experiments find that recurrence between middle layers can outperform recurrence through the full network. Training approximates temporal states with a fixed number of Jacobi iterations and controls backward depth separately. RLT uses feedback through the entire decoder and takes full-history replay with full BPTT as the reference, with optional TBPTT.

Latent recurrent language models. Latent Recurrent Transformer (LRT) reuses a high-level state from the previous token through KV projection and residual injection, keeping a decoder-only backbone and one forward pass per generated token (Huang et al., 2026). Its interleaved parallel training refines subsets of positions from a shared state buffer. For RL, it initializes this buffer with detached rollout states to reduce differences between rollout and recomputation. RLT separates encoder memory from the recurrent decoder and rebuilds the full history under current parameters for exact policy replay.

Continuous latent computation. Coconut feeds the last hidden state back as the next input embedding during a latent reasoning phase, using a curriculum that replaces textual reasoning steps with continuous states (Hao et al., 2024). PonderLM-2 inserts latent steps between ordinary tokens during pretraining and uses Jacobi iterations to approximate their recurrent dependencies in parallel (Zeng et al., 2025). RLT advances its state once per ordinary prompt or response token, without adding latent positions.

Block- and segment-level recurrence. Block-Recurrent Transformers update persistent state vectors with attention and gates, processing a block of tokens in parallel at each recurrent step (Hutchins et al., 2022). Recurrent Memory Transformer carries learned memory-token representations between sequence segments (Bulatov et al., 2022). RLT updates its final decoder output and SWA caches at every token, alongside separate encoder memory. Its finer update interval requires sequential decoder work within each block.

Weight sharing across depth. Universal Transformers share weights across depth (Dehghani et al., 2018), and recurrent-depth latent reasoning repeatedly applies a recurrent block to increase computation (Geiping et al., 2025). RLT advances its state across tokens and optionally shares compatible encoder and decoder weights.

9 Conclusion

RLT carries a decoder hidden state and SWA caches across every prompt and response token. Its recurrent path grows with the sequence, while encoder memory can be built separately and reused. The same transition defines training, sampling, and replay under current parameters.

The eight-layer experiments show earlier parity learning for some encoder–decoder splits and a large single-seed accuracy difference on flat modular arithmetic. The relative performance of the splits varies across tasks. The next experiments will compare RLT and RLT w/o feedback under matched training settings to isolate hidden-state feedback.

References

- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer. *arXiv preprint arXiv:2207.06881*, 2022. URL <https://arxiv.org/abs/2207.06881>.
- Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T²MLR: Transformer with temporal middle-layer recurrence. *arXiv preprint arXiv:2607.15178*, 2026. URL <https://arxiv.org/abs/2607.15178>.
- Mia Xu Chen, Orhan Firat, Ankur Bapna, Melvin Johnson, Wolfgang Macherey, George Foster, Llion Jones, Niki Parmar, Mike Schuster, Zhifeng Chen, Yonghui Wu, and Macduff Hughes. The best of both worlds: Combining recent advances in neural machine translation. *arXiv preprint arXiv:1804.09849*, 2018. URL <https://arxiv.org/abs/1804.09849>.
- DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report, DeepSeek-AI, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/resolve/main/DeepSeek_V41_Tech_Report.pdf. Sections 2.2 and 3.2.2; publicly available from the official DeepSeek model repository.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. *arXiv preprint arXiv:1807.03819*, 2018. URL <https://arxiv.org/abs/1807.03819>.

- Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*, 2020. URL <https://arxiv.org/abs/2002.09402>.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025. URL <https://arxiv.org/abs/2502.05171>.
- Riccardo Grazi, Julien Siems, Arber Zela, Jörg K. H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking state-tracking in linear rnns through negative eigenvalues. *arXiv preprint arXiv:2411.12537*, 2024. URL <https://arxiv.org/abs/2411.12537>.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024. URL <https://arxiv.org/abs/2412.06769>.
- Zeyi Huang, Xuehai He, Liliang Ren, Yiping Wang, Baolin Peng, Hao Cheng, Shuohang Wang, Pengcheng He, Jianfeng Gao, Yong Jae Lee, and Yelong Shen. Latent recurrent transformer: Architecture exploration, training strategies, and scaling behavior. *arXiv preprint arXiv:2605.26797*, 2026. URL <https://arxiv.org/abs/2605.26797>.
- DeLesley Hutchins, Imanol Schlag, Yuhuai Wu, Ethan Dyer, and Behnam Neyshabur. Block-recurrent transformers. *arXiv preprint arXiv:2203.07852*, 2022. URL <https://arxiv.org/abs/2203.07852>.
- Costin-Andrei Oncescu, Depen Morwani, Samy Jelassi, Alexandru Meterez, Mujin Kwun, and Sham Kakade. The recurrent transformer: Greater effective depth and efficient decoding. *arXiv preprint arXiv:2604.21215*, 2026. URL <https://arxiv.org/abs/2604.21215>.
- Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*, 2024. URL <https://arxiv.org/abs/2405.05254>.
- Xi Wang, Ziyang Cai, Zheng Zhan, Harry Dong, Ying Fan, Gustavo de Rosa, Tim Pearce, and John Langford. Full-bandwidth transformer. *arXiv preprint arXiv:2608.08888*, 2026. URL <https://arxiv.org/abs/2608.08888>.
- Boyi Zeng, He Li, Shixiang Song, Yixuan Wang, Ziwei He, Xinbing Wang, and Zhouhan Lin. PonderLM-2: Pretraining LLM with latent thoughts in continuous space. *arXiv preprint arXiv:2509.23184*, 2025. URL <https://arxiv.org/abs/2509.23184>.
- Yifan Zhang et al. Reliable RL scaling requires accounting for Prefill–Decode kernel mismatch. Technical report, Pretraining-RL-Science project, August 2026. URL https://github.com/yifanzhang-pro/Pretraining-RL-Science/blob/master/Prefill_Decode_Kernel_Mismatch.pdf. Dated August 6, 2026; revised August 24, 2026.

Appendix

A	Execution Procedures	21
A.1	Recurrent prompt prefill	21
A.2	RLT without hidden-state feedback	21
A.3	Generation and new external inputs	21
A.4	Pretraining and SFT	22
A.5	Current-policy RL replay	22
B	Causality and Gradient Paths	23
C	When Cached States Can Be Reused	24
D	Experimental Details	24
D.1	Parameter counts	24
D.2	Implementation of RLT w/o feedback	25
D.3	Seed aggregation and metric definitions	25
D.4	Reproducing the snapshot	25

A Execution Procedures

The complete decoder state is $H_t = (s_t, C_t^D)$, with $H_0 = (s_*, \emptyset)$. Encoder continuation state and encoder-derived cross-attention memory are maintained separately. All schedules use the block order and window convention in Equation (2.16).

A.1 Recurrent prompt prefill

1. Start an independent sequence with $x_1 = \text{BOS}$ and $H_0 = (s_*, \emptyset)$; initialize encoder caches and positions.
2. Encode $x_{1:T}$ causally in parallel and construct its encoder KV memory.
3. For $t = 1, \dots, T$, compute $H_t = D_\phi(\text{Merge}(e_t, s_{t-1}); M_{\leq t}, C_{t-1}^D, t)$. Each decoder layer reads current KV and the retained SWA history. Both attention masks exclude positions after t .
4. Predict the first response token from s_T . Preserve H_T , encoder caches, memory, and positional metadata for the next update.

Each decoder update produces the KV needed by later updates, so this loop remains sequential even when the encoder runs in parallel.

A.2 RLT without hidden-state feedback

RLT w/o feedback removes feedback of the final decoder hidden state (Figure 6). It retains the causal encoder, encoder memory, decoder SWA, and prefix-restricted memory attention. The learned initial state s_* , state normalization, gated merge, and feedback projection are removed. Each decoder position takes e_t as input in place of $\text{Merge}(e_t, s_{t-1})$.

For known tokens, decoder positions run together within each layer under causal SWA and encoder-prefix masks. The resulting layerwise KV can be saved for incremental continuation. Generation remains sequential because future tokens are unknown. Full-sequence training uses ordinary backpropagation without a recurrent TBPTT boundary.

The control tests hidden-state feedback with the encoder–decoder split and attention structure held fixed.

A.3 Generation and new external inputs

Sample x_{t+1} from the distribution predicted by s_t , then consume it by updating the encoder cache and memory and advancing the complete decoder state to H_{t+1} . The resulting state predicts x_{t+2} . Every consumed token receives exactly one recurrent update and one KV insertion at every decoder SWA layer.

When a user or tool supplies multiple tokens, their encoder representations can be computed in a causal batch conditioned on the existing prefix. Decoder updates still occur in token order from the saved complete state. External tokens update both s_t and C_t^D .

If generation stops at a length limit before consuming its last emitted token, record that token separately. The cache represents the prefix already consumed; process the pending token once before any later token when resuming.

A.4 Pretraining and SFT

1. Compute $e_{1:S-1}$ using a causal encoder and construct memory.
2. Initialize $H_0 = (s_*, \emptyset)$; unroll all positions $1, \dots, S - 1$, including every layerwise SWA cache update.
3. Accumulate all valid next-token losses for pretraining, or only assistant-target losses for SFT. All context tokens receive state updates.
4. Normalize each example by its number of selected targets, then average examples as in Equations (5.1) and (5.2). Backpropagate through the complete computation, using checkpointing if required.

Examples without selected targets are excluded from the loss average. Token averaging would give more weight to examples with more selected targets.

Packed independent sequences need separate recurrent states, SWA caches, encoder caches, positions, and attention masks. Both attention mechanisms must stay within document boundaries, even where losses are masked. Training cache operations must retain gradient dependencies or allow equivalent recomputation.

A.5 Current-policy RL replay

1. Read the rollout token history, action mask, actual behavior log-probabilities, and sampling metadata. Behavior probabilities include all sampling transforms and renormalization.
2. Hold current parameter values fixed throughout forward replay and backward. Use the target policy’s positional, SWA, and execution conventions, with parameter gradients enabled.
3. Recompute encoder representations of the known history. From $H_0 = (s_*, \emptyset)$, rebuild the recurrent output and every decoder SWA cache through all prompt tokens.
4. Replay subsequent tokens in order. Before consuming each sampled action, read its current-policy log-probability from the preceding state. Include EOS if sampled. Consume every intervening external token needed for later predictions.
5. Form the chosen RL loss from action log-probabilities, rewards or advantages, and any required behavior ratios. Apply action factors only to tokens sampled by the policy.
6. Backpropagate, update parameters, and invalidate parameter-dependent encoder and decoder caches before the next exact current-policy replay.

The action mask selects loss terms while preserving gradients through user and tool tokens. Replay all assistant turns with the external context between them. Prompt replay under `no_grad`, or detaching prompt KV, can keep forward values unchanged while dropping gradients required by full BPTT.

Importance weights must satisfy the support conditions described with Equation (5.5).

A deterministic forward pass, for example with dropout disabled, gives a reproducible probability for each action. For a stochastic policy, specify whether action probabilities condition on internal randomness or average over it. One replay evaluates a particular realization; marginal probabilities require averaging over that randomness.

B Causality and Gradient Paths

Proposition B.1 (Causality). With a causal encoder, prefix-restricted encoder memory, and causal decoder SWA, $H_t = (s_t, C_t^D)$ depends only on $x_{1:t}$ and the parameters, including s_* , under deterministic execution.

Proof. Encoder causality implies that each e_j depends only on $x_{1:j}$, hence $M_{\leq t}$ depends only on $x_{1:t}$. The initial state contains no future-token information. Suppose H_{t-1} depends only on $x_{1:t-1}$. The next update uses this state, e_t , and $M_{\leq t}$. At each decoder layer, current KV is formed from the causally available layer input, and SWA reads no position greater than t . Appending current KV and evicting old entries introduce no future information. Thus both s_t and C_t^D depend only on $x_{1:t}$, completing the induction. $\square$

For teacher-forced encoder features held fixed, use a fixed-slot representation of the decoder cache, with validity masks during warm-up, and define

$$\mathcal{J}_t = \frac{\partial H_t}{\partial H_{t-1}}. \quad (\text{B.1})$$

Then

$$\frac{\partial H_t}{\partial H_j} = \mathcal{J}_t \mathcal{J}_{t-1} \cdots \mathcal{J}_{j+1}, \quad j < t, \quad (\text{B.2})$$

where

$$\mathcal{J}_t = \begin{pmatrix} \frac{\partial s_t}{\partial s_{t-1}} & \frac{\partial s_t}{\partial C_{t-1}^D} \\ \frac{\partial C_t^D}{\partial s_{t-1}} & \frac{\partial C_t^D}{\partial C_{t-1}^D} \end{pmatrix}. \quad (\text{B.3})$$

The block Jacobian includes paths through both the recurrent output and decoder KV.

Encoder memory retains information from past encoder outputs; decoder SWA gives direct access to recent decoder projections. Older decoder entries are evicted, but may still affect later computation through states or activations that read them before eviction.

Full parameter derivatives also include the parameter dependence of encoder features, memory, every transition, and decoder KV projections. Let B_T^E denote all encoder-side boundary tensors needed for response continuation, including encoder continuation KV and encoder-derived cross-attention memory. For a response loss $\ell(\Theta, H_T(\Theta), B_T^E(\Theta))$, the chain rule gives

$$\frac{d\ell}{d\Theta} = \frac{\partial \ell}{\partial \Theta} + \frac{\partial \ell}{\partial H_T} \frac{\partial H_T}{\partial \Theta} + \frac{\partial \ell}{\partial B_T^E} \frac{\partial B_T^E}{\partial \Theta}. \quad (\text{B.4})$$

The first term holds the boundary arguments fixed; the other terms account for their prefix computation. In particular,

$$\frac{\partial \ell}{\partial H_T} \frac{\partial H_T}{\partial \Theta} = \frac{\partial \ell}{\partial s_T} \frac{\partial s_T}{\partial \Theta} + \frac{\partial \ell}{\partial C_T^D} \frac{\partial C_T^D}{\partial \Theta}. \quad (\text{B.5})$$

Detaching s_T , decoder KV, or encoder boundary tensors removes the corresponding terms from the full gradient, even when forward probabilities stay unchanged.

C When Cached States Can Be Reused

A saved prefix can replace forward replay when it matches the current parameters, consumed tokens, positions, initialization, SWA window, and policy execution settings. Save the recurrent output, each decoder SWA cache, encoder continuation state, encoder memory, and associated metadata. Handle stochastic computation as specified by the policy.

A detached cache can hold correct values without the computation graph needed for training. Full BPTT requires retaining or recomputing the prefix graph so gradients reach every parameter-dependent boundary tensor.

Activation checkpointing within one update recomputes activations under the same parameters and stochastic state. Mutable cache implementations must restore the saved contents during recomputation and preserve activations that backward still needs.

Truncated BPTT instead preserves numeric values while cutting selected gradient dependencies. A complete decoder-state detach is

$$\tilde{H}_t = (\text{stopgrad}(s_t), \text{stopgrad}(C_t^D)). \quad (\text{C.1})$$

Detaching only s_t leaves possible paths through decoder KV; detaching only decoder KV leaves paths through the recurrent output. Encoder-side boundary tensors can provide additional paths across the same boundary.

SWA eviction removes old KV from future attention windows. Full BPTT still differentiates earlier computations that used those entries, so training may need more activation storage than the inference cache alone.

After a weight update, recompute the prefix values and gradient graph before the next exact training pass.

D Experimental Details

D.1 Parameter counts

Table 3 lists unique trainable parameters. Each RLT decoder block has both SWA and encoder-memory cross-attention; each Transformer block has one attention sublayer. RLT 8 + 0 applies the learned state projection and gated merge at every token.

Table 3 Unique trainable parameters in the eight-layer models. Each model has the same size across all six tasks.

Model	Parameters (M)	Relative to GPT 8
RLT 4+4	28.73	1.135×
RLT 5+3	28.20	1.114×
RLT 6+2	27.68	1.093×
RLT 7+1	27.15	1.073×
RLT 8+0	26.10	1.031×
Transformer 8	25.31	1.000×

D.2 Implementation of RLT w/o feedback

The `rlt_parallel_tiny` control was merged after the snapshot in [nanogptpro-dev PR #164](#). It uses an untied 4 + 4 encoder-decoder split, width 512, four attention heads, FFN width 1,365, vocabulary capacity 277, an SWA window of eight, one encoder-memory group, and width- μ P. It has 27,938,944 unique parameters.

The control removes the learned initial state, state normalization, merge gate, and feedback projection from recurrent RLT 4 + 4. It retains causal encoder attention, decoder SWA, and prefix-only attention to encoder memory. Known positions run in parallel within each decoder layer, and training uses full backpropagation without TBPTT. Generation maintains KV caches and emits tokens sequentially, without feeding the previous final hidden state into the next decoder input. The two models use separate checkpoint formats identified by their architecture markers.

D.3 Seed aggregation and metric definitions

Let $a_{m,s,t}$ be validation accuracy for model m , initialization seed s , and optimizer step t . For parity we report

$$\bar{a}_{m,t} = \frac{1}{3} \sum_{s \in \{42,43,44\}} a_{m,s,t}, \quad \text{SD}_{m,t} = \sqrt{\frac{1}{2} \sum_{s \in \{42,43,44\}} (a_{m,s,t} - \bar{a}_{m,t})^2}. \quad (\text{D.1})$$

We compute a mean only when all three seeds have a value at the same step. Missing checkpoints are left missing, so each parity curve ends at the last step shared by its seeds and always uses $n = 3$. Table 1 uses the latest step shared by all six model curves within each task. The SD describes initialization variability on the fixed data stream and validation set.

Parity and mod-5 each supervise one final label per example. Their three equally sized validation groups are pooled before averaging across seeds. S_5 supervises every prefix during training; final-state accuracy counts only the last state in each sequence. Table 4 also reports the mean fraction of correct prefix states. Addition accuracy requires an exact greedy answer and EOS, while its loss is teacher-forced cross-entropy normalized per example.

Table 4 Final-state / prefix-token accuracy (%) on the fixed S_5 validation sets at step 1,000, seed 42.

Task	RLT 4+4	RLT 5+3	RLT 6+2	RLT 7+1	RLT 8+0	GPT 8
S_5 , swaps	100.00 / 100.00	99.61 / 99.80	95.70 / 99.08	89.84 / 98.73	86.33 / 98.66	89.84 / 98.75
S_5 , standard	1.17 / 3.99	3.52 / 7.82	1.95 / 7.65	1.56 / 7.06	1.17 / 4.17	1.17 / 4.00

D.4 Reproducing the snapshot

The formal tasks use data seeds 20260914 for training, 20260915 for validation, and 20260916 for the separately configured test sets. Addition uses data seed 42. Training examples and held-out sets are fixed across models; the parity repeats vary only initialization. All measured runs use the implementation based on upstream commit d1a4516.

The files `data/experiments-depth8/manifest.json` and `configs.json` record source and corpus fingerprints, run identifiers, model settings, timestamps, and progress. Validation records, complete training histories, and aggregates at common steps are stored alongside them. The script `scripts/plot_depth8.py` regenerates the figures and tables.

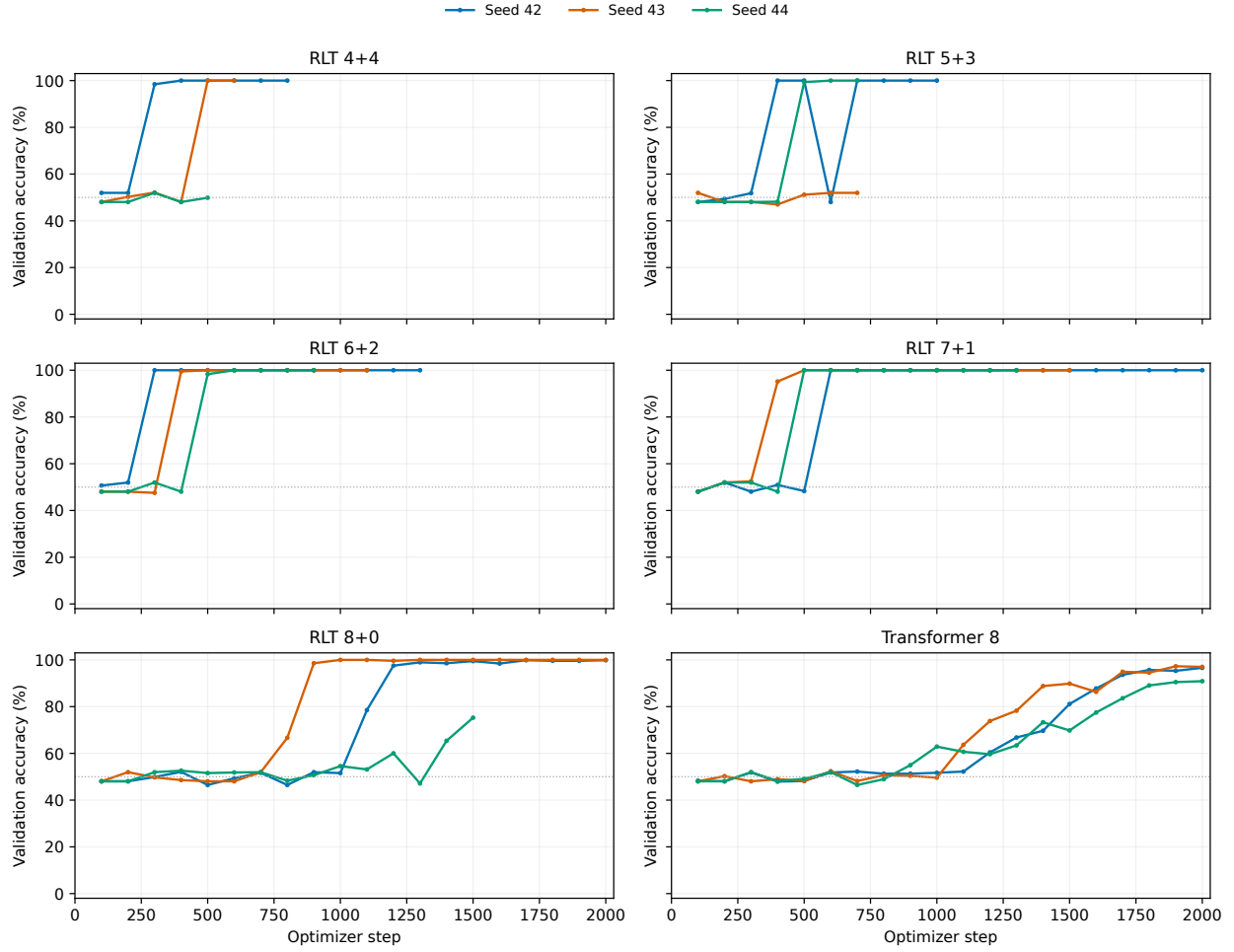


Figure 10 Parity accuracy for each initialization seed. Curves include checkpoints beyond the end of the three-seed mean and stop at each run’s latest validation. All runs share the training stream and validation set.